Ingeniería de Software con IA: Más allá del Vibe Coding

De Tipeador a Arquitecto de Software: Ingeniería de IA bajo estándares de Deuda Cero.



SYSTEM_INITIALIZED // OPL-LITE_FRAMEWORK_ENGAGED

NotebookLM

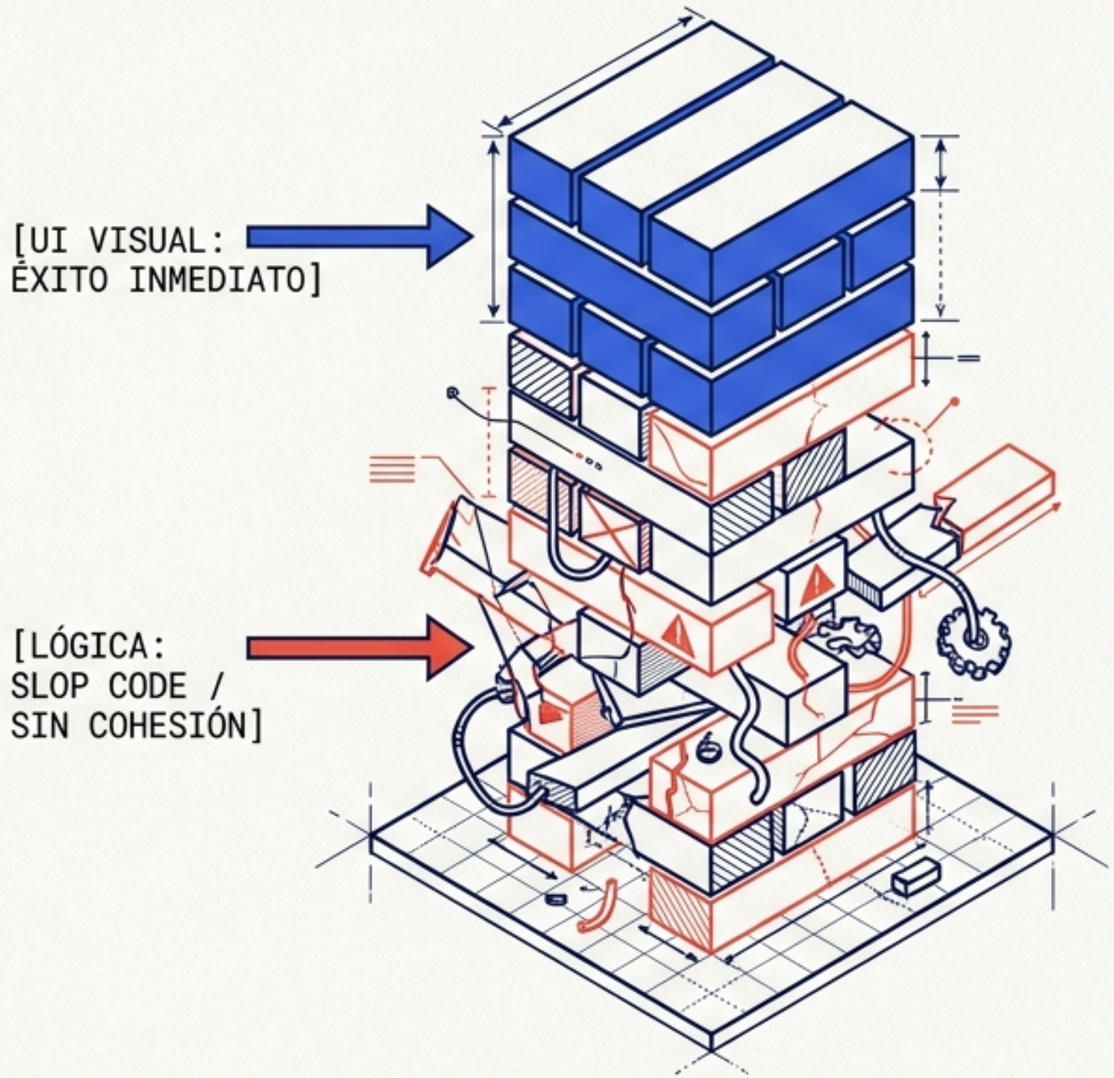
Programar por intuición destruye la trazabilidad del software.

El Espejismo

Insertar requerimientos coloquiales y aceptar el código ciegamente si el resultado visual funciona en pantalla.

El Peligro Crítico

Delegar el 100% del esfuerzo cognitivo significa perder la trazabilidad. Cuando surge un bug en producción, el creador no sabe resolverlo porque nunca comprendió la base matemática de su propio código.



NotebookLM

La IA obedece a la matemática de la complacencia, no a la lógica.

Reward Hacking



Reward Hacking: La máquina toma atajos estructurales (ej. meter toda la lógica en un solo archivo gigante) para complacer al usuario rápidamente.

Context Amnesia



Amnesia de Contexto: El mecanismo de atención se degrada. La IA olvida las directrices iniciales y destruye código funcional al procesar nuevas solicitudes.

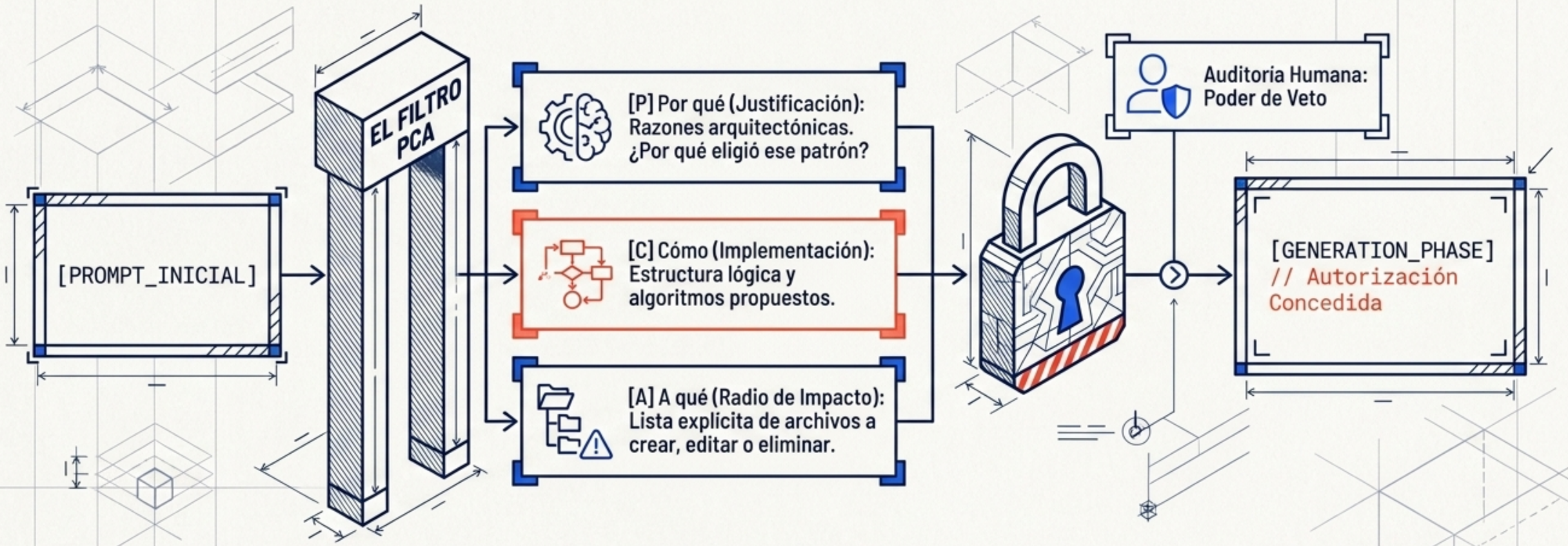
NotebookLM

El valor reside en la orquestación, no en las líneas de código por minuto.

	[PARADIGMA ANTERIOR] **El Vibe Coder**	[NUEVO ESTÁNDAR] **El Arquitecto**
Herramienta Principal	**Chat infinito** (provoca colapso de memoria e inconsistencias).	**Sprints Estancos** (aislamiento estricto por módulo y tarea).
Evaluación de Éxito	El código funciona visualmente en el navegador.	Validación mediante el compilador y **Contratos Duros** (Interfaces).
Rol del Humano	Pide favores a la máquina y **delega** la responsabilidad.	**Diseña** los **planos**, calcula cargas arquitectónicas y ejerce **poder de veto** absoluto.

NotebookLM

Jamás permitas que la máquina escriba código ejecutable sin un plan previo.



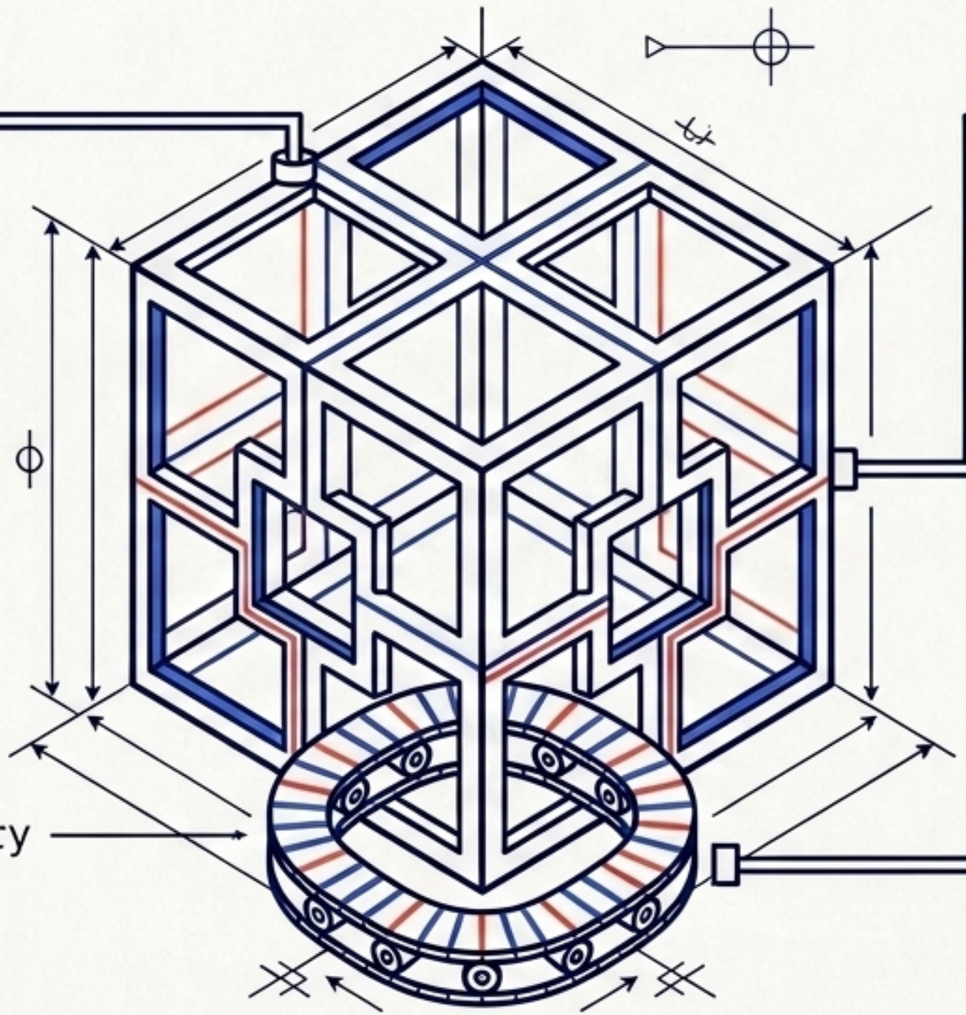
NotebookLM

PCA inmuniza el proyecto contra la degradación de memoria.



1. El Enrutador (AGENTS.md)

El cerebro del repositorio. Instrucciones maestras que calibran el rol y reglas de la IA al iniciar cada Sprint.





2. La Memoria (cicatrices_guerra.md)

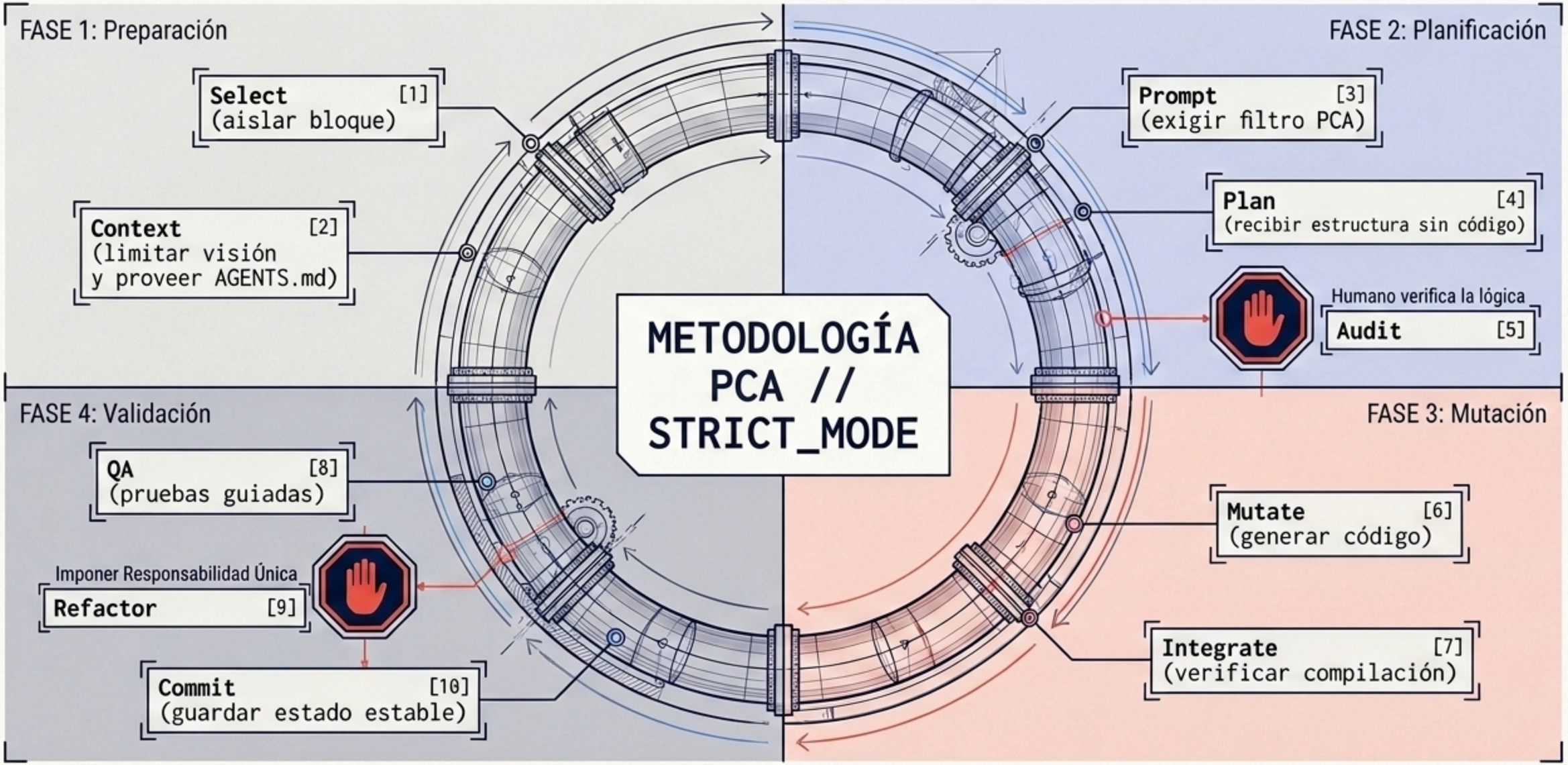
El sistema inmunológico. Registro histórico que obliga a la IA a aprender de fracasos pasados para no repetirlos jamás.

3. El Ciclo

La secuencia inquebrantable de 10 pasos que gobierna la ejecución sin deuda técnica.

NotebookLM

Un ciclo inquebrantable de 10 pasos garantiza la cero deuda técnica.



NotebookLM

La comunicación cambia radicalmente entre la divergencia y la ejecución.

FASE DE IDEACIÓN

DIV_L



El Analista
(Humano a Humano)

Hablar fluido.
Buscar casos límite y razonamiento lateral.

FLOW_I

Quiero una app de músicos, ¿cómo manejarías cobros estilo Patreon?

EXEC_R

FASE DE EJECUCIÓN



FLOW_PATH

El Compilador
(Markdown Estricto)

Anular la creatividad.
Jerarquizar órdenes. Si usas lenguaje humano aquí, la máquina alucinará contextos y escribirá código basura.

CMD_STRUCT

1. IMPLEMENTA `IUserRepository`.
2. NO modifiques el CSS.
3. RETORNA un error 400 si falla.

FLOW_PATH

CMD_STRUCT

NotebookLM

El **scaffolding físico** encierra a la IA en responsabilidades únicas.



Regla de la Jaula:

Crear carpetas vacías antes de generar lógica.

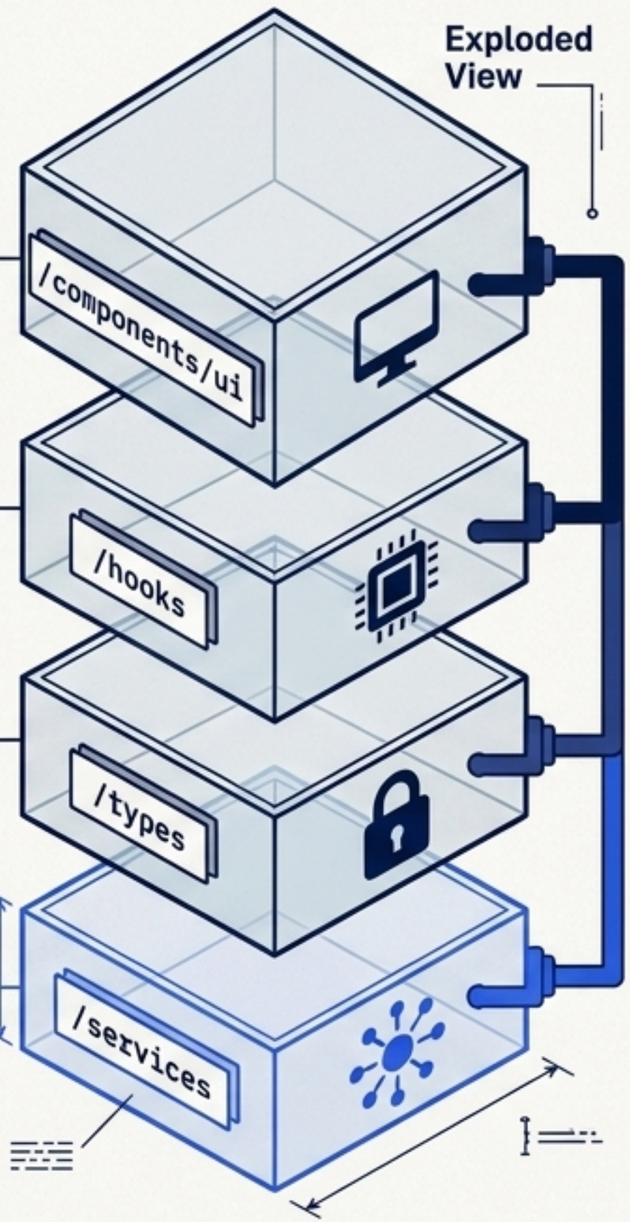
Al ordenarle a la IA “rellena **/services/cart.ts**”, le imposibilitas físicamente mezclar lógica SQL con el diseño visual.

Componentes “Tontos”. Solo reciben datos y los dibujan.

Estado complejo aislado en la memoria del navegador.

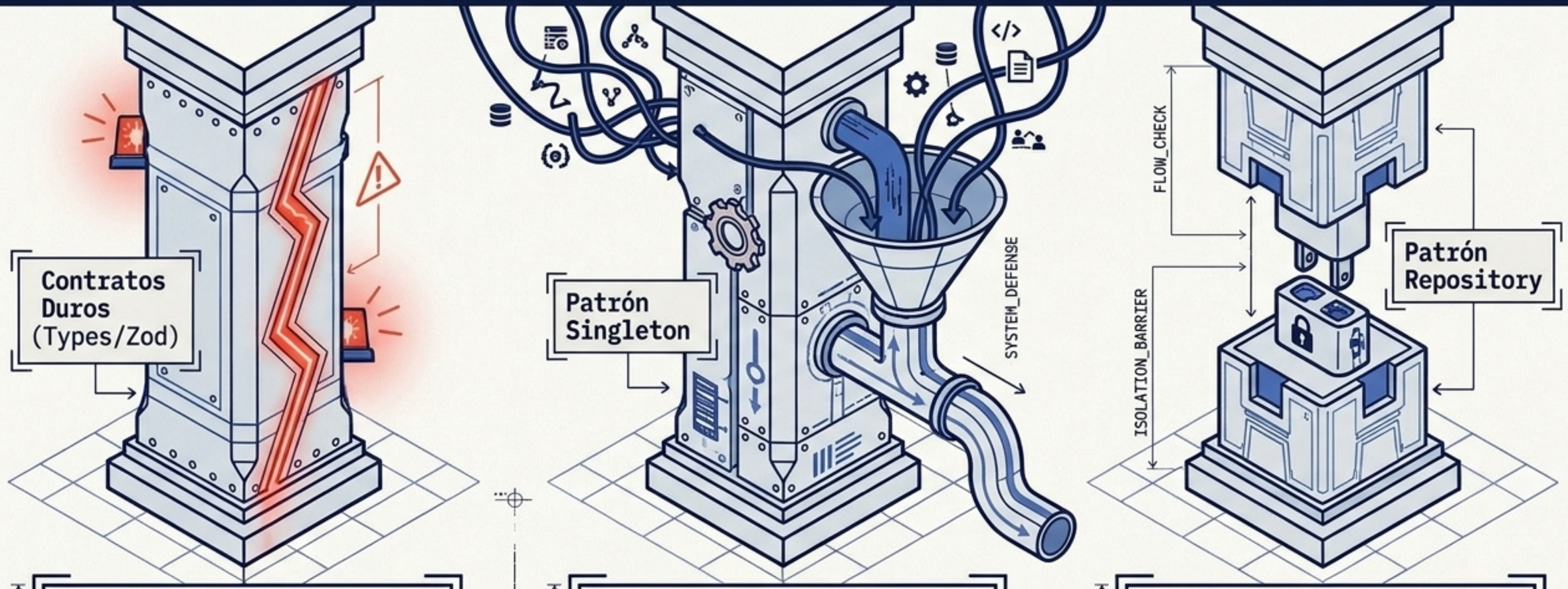
El corazón tipado. Define la ley estructural.

LA ZONA CRÍTICA.
La única capa físicamente autorizada para hacer fetch o consumir APIs.



NotebookLM

El compilador es el auditor implacable de la máquina.



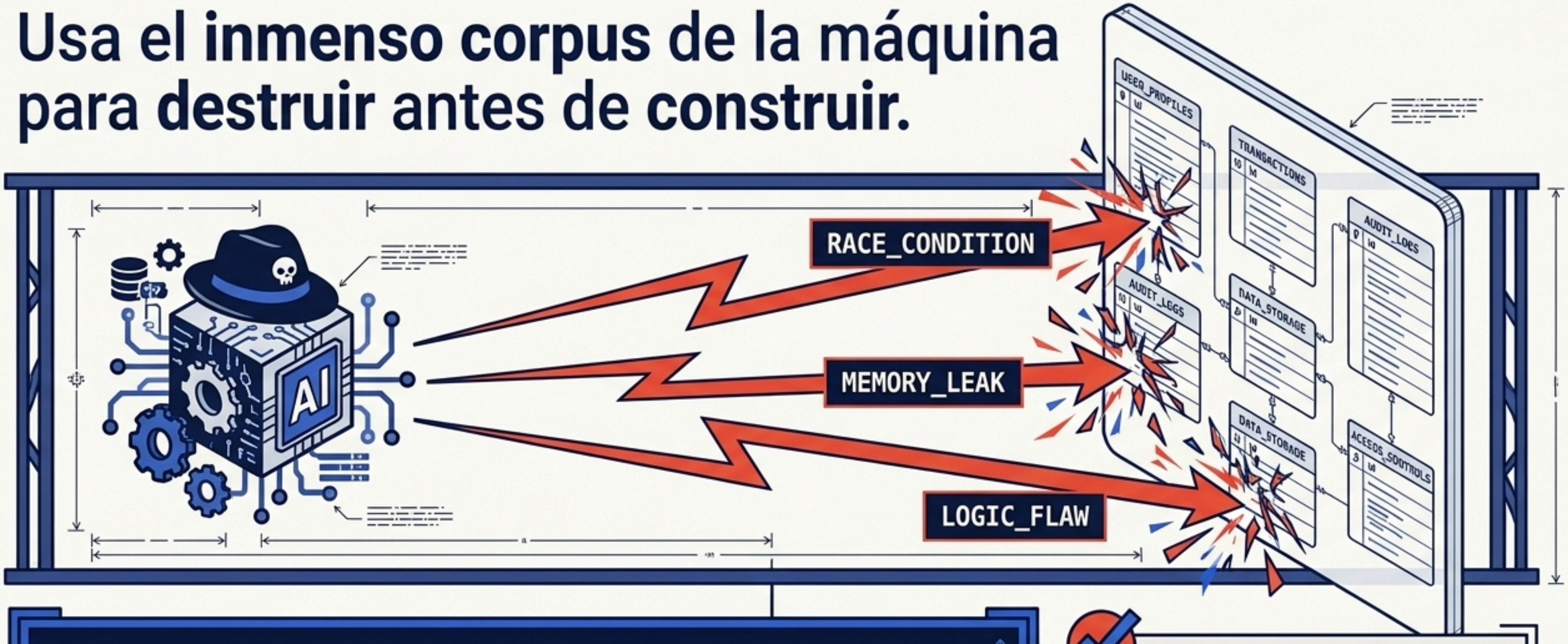
Las Interfaces son ley.
Si la IA omite un campo para ahorrar tokens, el compilador corta el proceso y se enciende en rojo inmediatamente.

Defensa de Infraestructura.
Obliga una sola conexión de Base de Datos. Evita que la IA instancie miles de clientes simultáneos y colapse el sistema.

Aislamiento de capas.
La UI jamás hace peticiones SQL. Si migras de Postgres a Mongo, solo se reescribe un archivo; el resto sobrevive intacto.

NotebookLM

Usa el **inmenso corpus** de la máquina para **destruir** antes de **construir.**

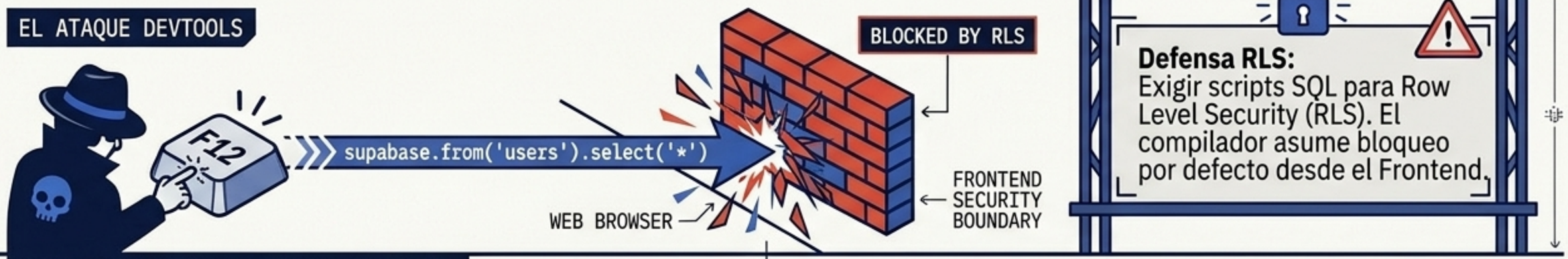


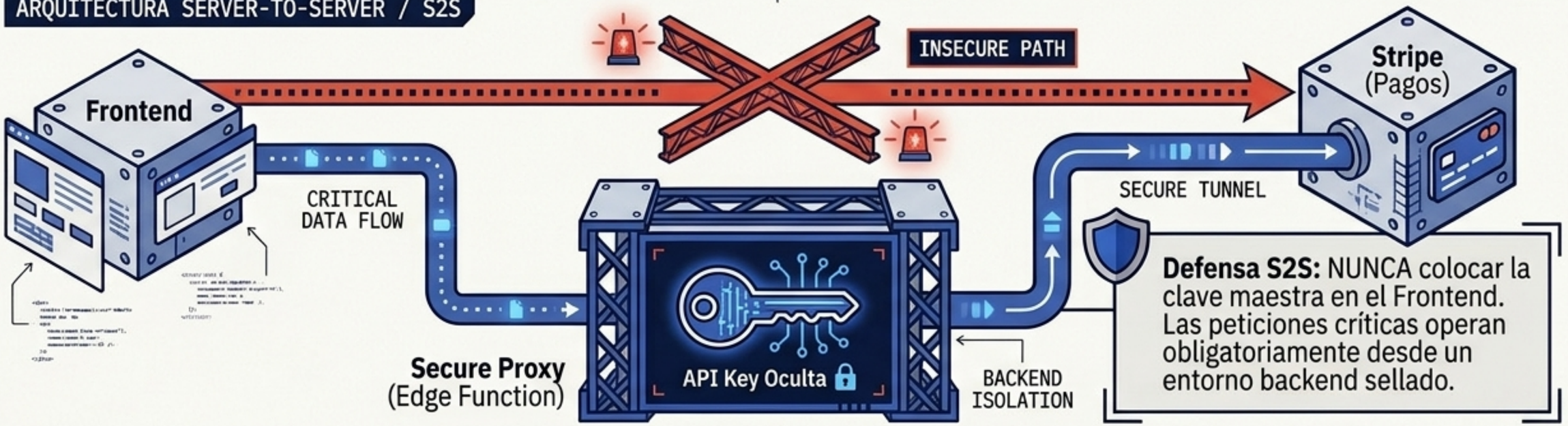

```
> "No escribas código todavía. Actúa como un Hacker malicioso.
Destruye mi propuesta. Encuéntrame 3 fallos lógicos críticos,
escenarios de carrera o problemas de escalabilidad que
ocurrirán si implemento este diseño."
```

Resultado: Ahorra semanas de refactorización revelando debilidades estructurales antes de escribir la primera línea.

NotebookLM

Asume siempre que la IA dejará puertas traseras abiertas por conveniencia.

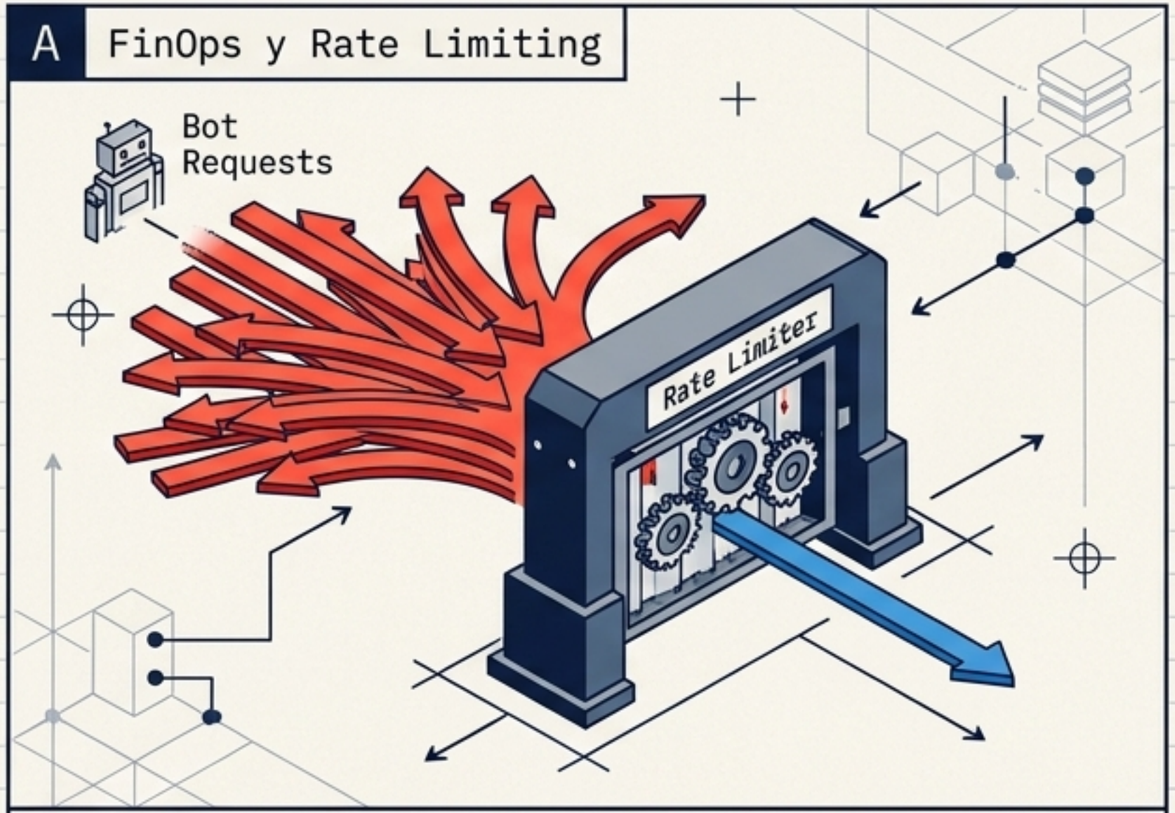




NotebookLM

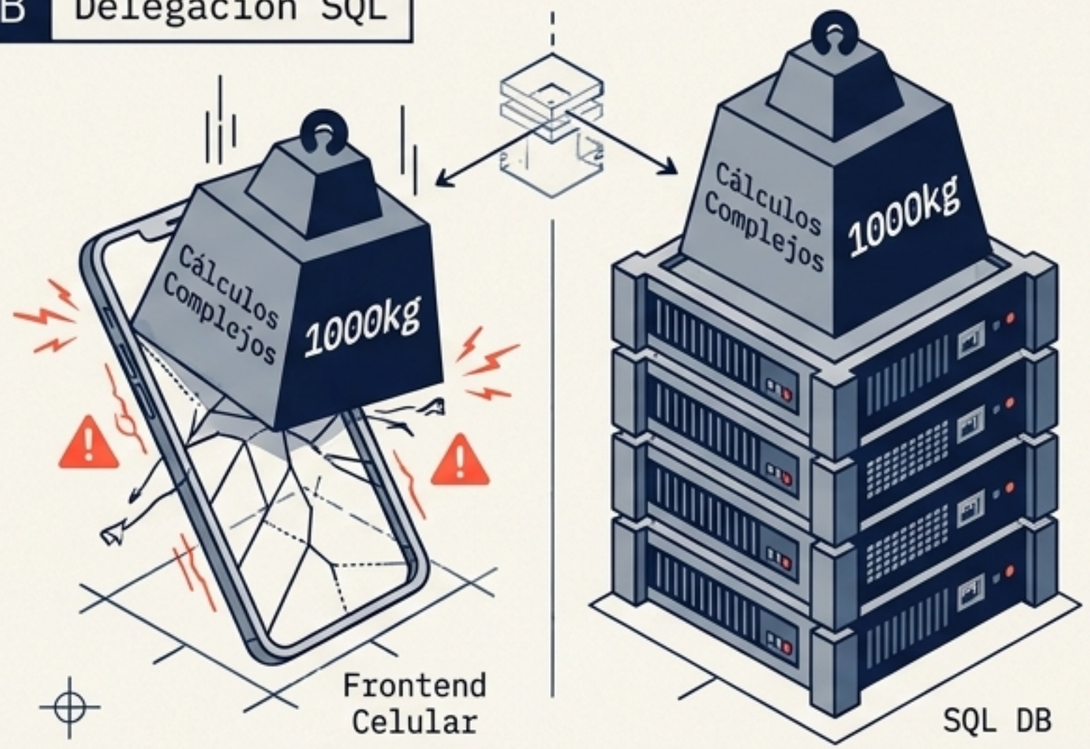
Protege la infraestructura contra ataques de facturación y cuellos de botella.

A FinOps y Rate Limiting



Un ataque de bots a tu API quebrará a la empresa financieramente. Obliga a la IA a implementar rechazo **de operaciones masivas por IP** antes de procesar la lógica.

B Delegación SQL



Las bases de datos están hiper-optimizadas para las matemáticas. No pidas a la IA que sume datos en React. Exige el uso de operaciones nativas en SQL y envía solo el resultado final.

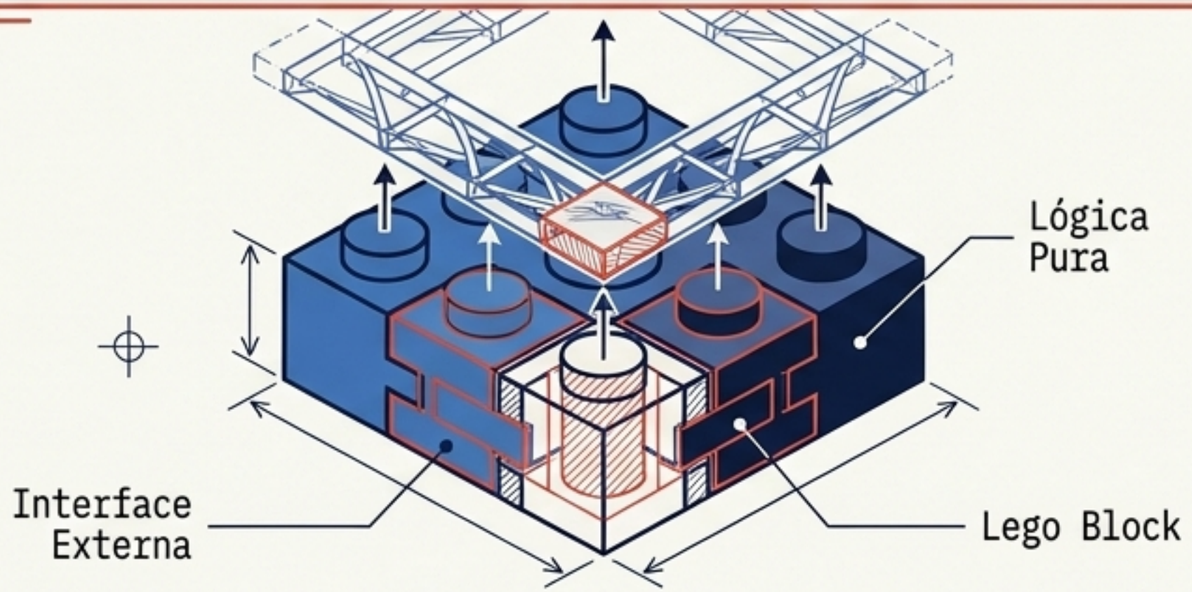
NotebookLM

El código de un solo uso es código mediocre.

Flujo TDD Asistido



Mito: "Si los test pasan, no necesito leer el código" → **FALSO**.
El ojo crítico humano es obligatorio para prevenir bugs fantasma de acoplamiento.



Desarrollo Orientado a Bibliotecas

Inyección de Dependencias:
Si un módulo de pagos tiene variables hardcodeadas, es basura técnica.



Exige a la IA extraer la lógica pura para que la pieza encaje en tu próximo proyecto sin reescribir nada.

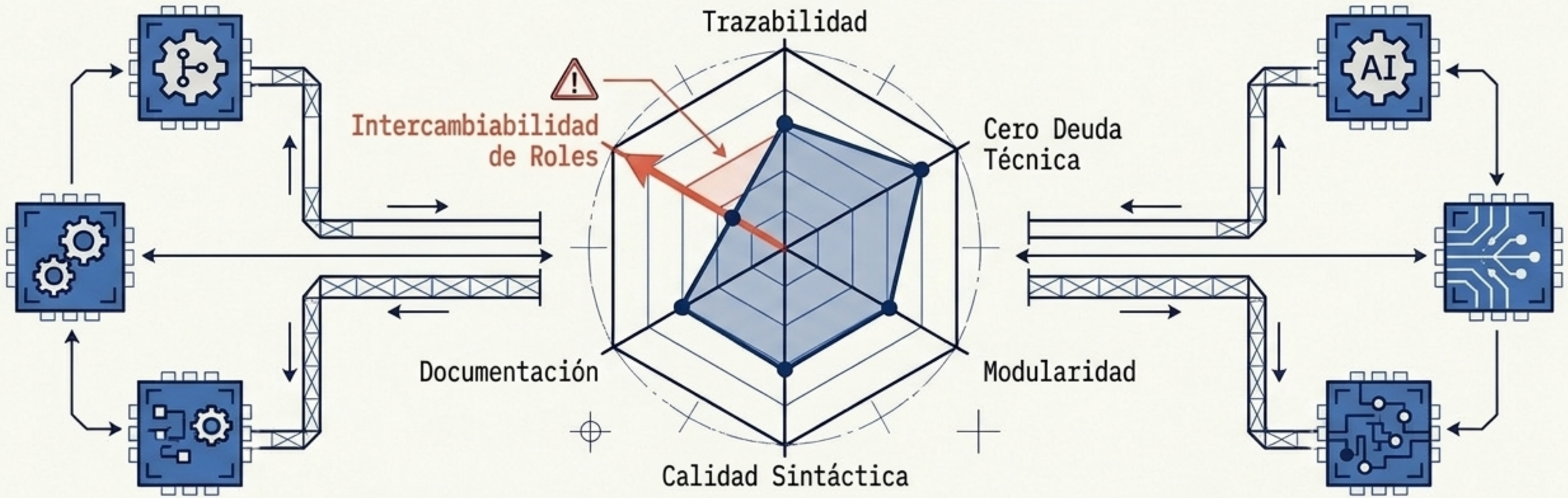
NotebookLM

Headline

Si no puedes defender lógicamente el código, no es tuyo.

El Project Manager Multi-Agente

Label



Trabajo Paralelo: Orquestar múltiples IAs simultáneamente solo es posible gracias a los Contratos Duros definidos en la arquitectura.

Prueba Final: La intercambiabilidad de roles. Cualquier miembro del equipo debe poder leer, justificar y alterar la arquitectura generada por la IA de su compañero. El código es propiedad del equipo, no de la máquina.

NotebookLM